

Cognizant Interim Preparation Guide - Spring Core, Boot and MVC

This PDF contains interview-oriented explanations, why concepts are used, architecture understanding, real-time examples, and coding examples for the topics visible in your curriculum screenshot. Prepared for Cognizant Interim preparation.

1. Dependency Injection (Setter Injection, Constructor Injection, Autowiring, Collections, Inner Beans, Bean Scope)

What is Dependency Injection?

Dependency Injection (DI) is a design pattern where Spring provides objects automatically instead of the programmer manually creating them.

Why do we use DI? Reduces tight coupling between classes Improves code maintainability Makes testing easier

Promotes reusable code **Real-time Example:**

A Food Delivery App service depends on PaymentService. Instead of creating PaymentService manually, Spring injects it automatically.

```
@Component
class PaymentService {
    public void pay() {
        System.out.println("Payment successful");
    }
}

@Component
class OrderService {

    @Autowired
    PaymentService paymentService;

    public void placeOrder() {
        paymentService.pay();
    }
}
```

2. Spring AOP

What is Spring AOP?

Aspect Oriented Programming separates cross-cutting concerns like logging, security, and transactions.

Why use AOP? Reduces duplicate code Centralized logging Improves maintainability **Real-time Example:**

Logging every API request in an E-commerce application.

```
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.example.service.*(..))")
    public void logBefore() {
        System.out.println("Method execution started");
    }
}
```

3. Reading Values from Property Files

What is application.properties?

It stores configuration values like database URL, username, password, and server port.

Why use it? Centralized configuration management Easy environment changes Improves maintainability

```
application.properties

server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/testdb

@Value("${server.port}")
private String port;
```

4. Spring JDBC and Transactions

What is Spring JDBC?

Spring JDBC simplifies database operations using JdbcTemplate.

Why use Spring JDBC? Reduces boilerplate code Handles exceptions automatically Improves database interaction

```
@Autowired
JdbcTemplate jdbcTemplate;

public void saveEmployee() {
    jdbcTemplate.update(
        "insert into employee(id,name) values(?,?)",
        1, "Anvetha"
    );
}
```

5. Spring MVC Fundamentals

What is Spring MVC?

Spring MVC is used for developing web applications using Model, View, and Controller architecture.

Flow: Client -> Controller -> Service -> DAO -> Database

Why use MVC? Separation of concerns Easy maintenance Better scalability

```
@Controller
public class HomeController {

    @GetMapping("/home")
    public String home(Model model) {
        model.addAttribute("msg", "Welcome");
        return "home";
    }
}
```

6. Spring Boot

What is Spring Boot?

Spring Boot simplifies Spring application development using auto-configuration and embedded servers.

Why use Spring Boot? No XML configuration Embedded Tomcat Rapid development Production ready features

```
@SpringBootApplication
```

```

public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}

```

7. Spring Data JPA

What is Spring Data JPA?

Spring Data JPA simplifies database operations using repository interfaces.

Why use JPA? Less SQL writing Easy CRUD operations Supports relationships **Relationships:** One-to-One, One-to-Many, Many-to-One, Many-to-Many

```

@Entity
public class Employee {

    @Id
    private int id;

    private String name;
}

@Repository
public interface EmployeeRepo extends JpaRepository<Employee,Integer> {
}

```

8. JPQL and Criteria Query

What is JPQL?

JPQL is Java Persistence Query Language used to query entity objects instead of tables.

Criteria Query:

Used for dynamic queries in JPA.

```

@Query("select e from Employee e where e.name=?1")
List<Employee> findByName(String name);

```

9. Spring REST

What is REST API?

REST APIs allow communication between frontend and backend using HTTP methods.

HTTP Methods: GET - Fetch data POST - Create data PUT - Update data DELETE - Delete data **Real-time Example:** Swiggy/Zomato mobile app communicates with backend APIs.

```

@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @GetMapping
    public List<Employee> getAll() {
        return service.getEmployees();
    }
}

```

10. Exception Handling and Validation

What is Exception Handling?

It handles application errors gracefully.

Why use Validation?

To ensure valid user input before processing.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public String handleException(Exception ex) {
        return ex.getMessage();
    }
}
```

Interview Tips: Explain concepts with real-time examples. Mention advantages and why the technology is used. Practice writing annotations from memory. Be confident while explaining architecture flow. Know differences between Spring, Spring MVC, and Spring Boot. Prepared specially for Cognizant Interim preparation.